

Lane Detection using Sobel Filter

1st Md Mostafizur Rahman

Electronic Engineering

Hochschule Hamm-Lippstadt

Lippstadt, Germany

md-mostafizur.rahman@stud.hshl.de

2nd Turja Barua

Electronic Engineering

Hochschule Hamm-Lippstadt

Lippstadt, Germany

turja.barua@stud.hshl.de

3rd Deepak Kapil

Electronic Engineering

Hochschule Hamm-Lippstadt

Lippstadt, Germany

deepak.kapil@stud.hshl.de

Abstract—This paper presents a complete hardware workflow for lane-edge detection, beginning with a VHDL prototype on a Nexys A7-100T board and ending with an Altium custom-PCB implementation. A fixed road image is converted to 8-bit grayscale, stored as an inferred FPGA ROM, streamed through a 3×3 Sobel operator, and displayed at 640×480 through VGA. Two implementations were completed: a 160×120 image-ROM baseline and an upgraded 320×240 version. The upgrade increases stored pixels by four, reduces nearest-neighbor display enlargement from 4× to 2×, and improves edge localization and visual detail. Vivado synthesis reports show that Block RAM utilization rises from 8 tiles (5.93%, displayed as 6%) to 24 tiles (17.78%, displayed as 18%), while LUT, flip-flop, I/O, and clock-buffer use remain small. The 320×240 implementation is therefore the highest practical resolution completed and verified in this project, not the absolute limit of the XC7A100T device. The custom PCB retains the XC7A100T-CSG324 FPGA and adds the required 1.0-V, 1.8-V, and 3.3-V rails, 100-MHz oscillator, JTAG, configuration flash, SW0 input, VGA output, and stabilization network. Supplied Altium outputs include the PCB document, BOM, Gerber layers, drill data, and a 3-D assembly view.

Index Terms—FPGA, VHDL, Sobel filter, lane detection, Artix-7, Nexys A7, image ROM, VGA, Block RAM, PCB, Gerber, BOM.

I. INTRODUCTION

Lane detecting extraction is a common pre-processing function in driver-assistance and autonomous-vehicle systems. A complete road-vision platform typically combines camera acquisition, color conversion, filtering, lane-model estimation, and temporal tracking. The objective of this Hardware Engineering Laboratory project is more focused: to demonstrate how a deterministic grayscale image can be processed pixel by pixel in FPGA hardware, and how the working evaluation-board prototype can be translated into a dedicated board concept.

The Sobel operator was selected because it estimates horizontal and vertical intensity gradients with two small 3×3 masks and can be implemented using additions, subtractions, shifts, line buffers, and registers. A live camera was intentionally avoided. Instead, a road image is converted offline to grayscale and stored in on-chip ROM. This makes the input repeatable, simplifies simulation, and allows the work to concentrate on VGA timing, memory addressing, streaming neighborhood construction, FPGA utilization, and PCB migration.

The project was developed in two stages. The first package stores 160×120 samples. The second stores 320×240 samples

while preserving the same 640×480 VGA timing and Sobel processing module. The second version is visually better because it contains four times as many source pixels and enlarges each source pixel by only two in both display dimensions. It is described as the maximum practical project resolution because it is the largest version actually implemented, synthesized, packaged, and compared within the submitted artifacts. It should not be interpreted as the theoretical maximum of the FPGA.

The main contributions are: 1) a modular four-file VHDL pipeline; 2) a streaming 3×3 Sobel implementation; 3) a quantitative 160×120 versus 320×240 resource comparison; 4) a systematic FPGA-to-PCB design method; and 5) documented schematic, layout, Gerber, BOM, drill, and 3-D deliverables.

TABLE I: Implemented Resolution Versions

	Baseline	Upgraded
ROM resolution	160×120	320×240
Stored pixels	19,200	76,800
Raw 8-bit storage	153,600 bit	614,400 bit
VGA output	640×480	640×480
Display enlargement	4× per axis	2× per axis
Completed bitstream	Yes	Yes

II. ARCHITECTURE AND VHDL DESIGN

The VHDL design contains four principal source files and one constraint file. `Lane_Detection_using_Sobel_Filter_top.vhd` integrates all modules and uses SW0 to select the original image or the Sobel result. `VGA_Display_640x480.vhd` generates horizontal and vertical counters, the visible interval, Hsync, Vsync, and a pixel-enable pulse. `image_rom_160x120.vhd`, or the corresponding 320×240 ROM, stores grayscale samples. `Sobel_Filter_3x3.vhd` forms a moving neighborhood with two previous-line memories and computes the edge magnitude. `Implemented_in_Artix7_100T.xdc` assigns the clock, switch, VGA buses, Hsync, and Vsync to device pins.

The 100-MHz board clock remains the only fabric clock. A two-bit divider asserts `pixel_ce` every fourth clock cycle, giving an effective 25-MHz pixel update. The VGA counters cover 800 horizontal clocks and 525 lines, of which 640×480

VHDL Lane Detection Block Diagram

Updated according to renamed source files

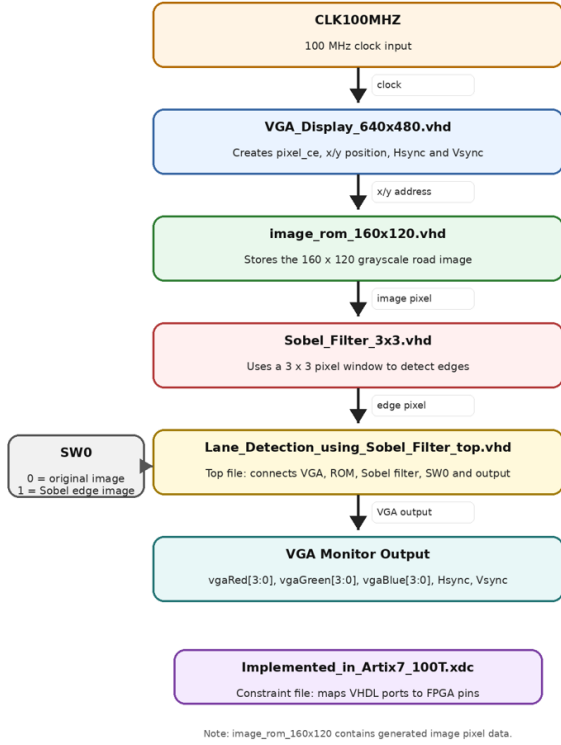


Fig. 1: Block Diagram for Lane Detection in FPGA.

are visible. During visible intervals, the screen coordinate is converted to an image-ROM coordinate. In the baseline, x (9 downto 2) and y (8 downto 2) divide by four. In the upgraded version, x (9 downto 1) and y (8 downto 1) divide by two, and the coordinate widths increase to cover 0–319 and 0–239. The VGA and Sobel files remain unchanged, demonstrating the benefit of modular design.

For a 3×3 grayscale window p_{ij} , the implemented directional gradients are

$$G_x = -p_{00} + p_{02} - 2p_{10} + 2p_{12} - p_{20} + p_{22}, \quad (1)$$

$$G_y = p_{00} + 2p_{01} + p_{02} - p_{20} - 2p_{21} - p_{22}. \quad (2)$$

The center column of the G_x mask is zero because horizontal-gradient evaluation compares the left and right sides. Similarly, the center row of the G_y mask is zero because vertical-gradient evaluation compares the upper and lower sides. The magnitude is approximated in hardware as

$$M = |G_x| + |G_y|, \quad (3)$$

which avoids a square root. Values above 255 are saturated to $x'FF$. Two line buffers retain previous rows, while three shift-register rows move the 3×3 window across the pixel stream. Control signals are delayed with the data so that the edge pixel, visible signal, Hsync, and Vsync remain aligned.

Verification was performed with module-level testbenches. A flat zero image was required to produce zero edge output. A known vertical step with columns 0, 0, and 255 produces

$G_x = 1020$, $G_y = 0$, and therefore a saturated output of 255. The original package also includes a ROM-address test and a full top-level simulation.

III. RESOLUTION COMPARISON AND VIVADO RESULTS

The 320×240 implementation stores 76,800 samples compared with 19,200 in the baseline. This fourfold increase provides more independent brightness samples along lane boundaries. When displayed at 640×480 , each upgraded sample occupies 2×2 screen pixels instead of 4×4 . Consequently, diagonal lane edges appear less block-like, the Sobel response is formed from finer spatial changes, and the visible edge position can be localized more accurately.

The upgrade is the highest practical resolution completed in this project for three reasons. First, it was successfully integrated into the existing VGA/Sobel architecture with only ROM and coordinate changes. Second, it improves display quality while using 17.78% of the available Block RAM tiles, leaving substantial device headroom for the processing logic and future functions. Third, a native 640×480 8-bit ROM would contain four times as many samples as 320×240 and would significantly increase source-file size, synthesis effort, and memory pressure. Thus, 320×240 is a balanced, verified project endpoint rather than an absolute silicon limit.

TABLE II: Vivado Synthesis Utilization Comparison

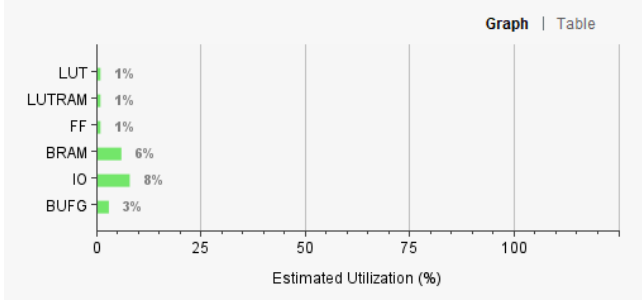
Resource	160×120	320×240
Slice LUTs	359 (0.57%)	372 (0.59%)
Flip-flops	165 (0.13%)	167 (0.13%)
BRAM tiles	8 (5.93%)	24 (17.78%)
DSP48E1	0 (0%)	1 (0.42%)
Bonded I/O	16 (7.62%)	16 (7.62%)
BUFG	1 (3.13%)	1 (3.13%)

The dominant scaling cost is Block RAM. The reported tile count triples from 8 to 24, so displayed utilization increases from approximately 6% to 18%, an increase of 11.85 percentage points. The source image contains four times as many pixels, but BRAM inference and packing cause a threefold tile increase rather than a perfect fourfold increase. LUT and flip-flop use remain below 1%, and the external I/O count is unchanged. One DSP block appears in the higher-resolution report, most likely because Vivado inferred arithmetic for the larger ROM address expression.

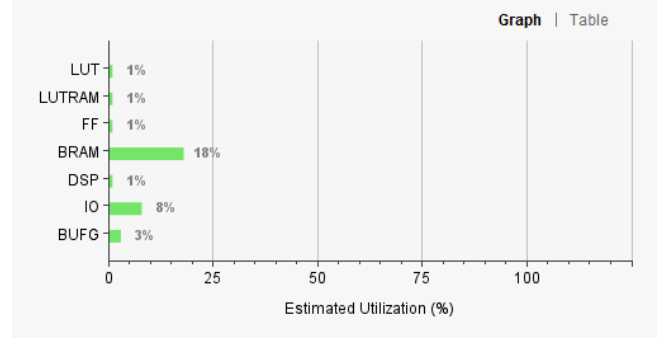
The comparison demonstrates that ROM capacity, rather than Sobel arithmetic, is the principal scaling factor. This is expected because the Sobel operator still processes one 3×3 window per pixel and its arithmetic width is unchanged. Only the stored frame grows. For future resolutions, an external memory or live pixel stream would be more scalable than embedding a complete frame as a large VHDL constant.

IV. FPGA TO CUSTOM PCB

The custom-PCB idea was obtained by separating the functions already provided by the Nexys A7 board from the functions implemented inside the FPGA. The Sobel algorithm, image ROM, VGA timing, and display selector remain VHDL



(a) Vivado Synthesis Utilization Report for 160×120.



(b) Vivado Synthesis Utilization Report for 320×240.

Fig. 2: Vivado utilization graphs showing the BRAM increase from 6% to 18%.

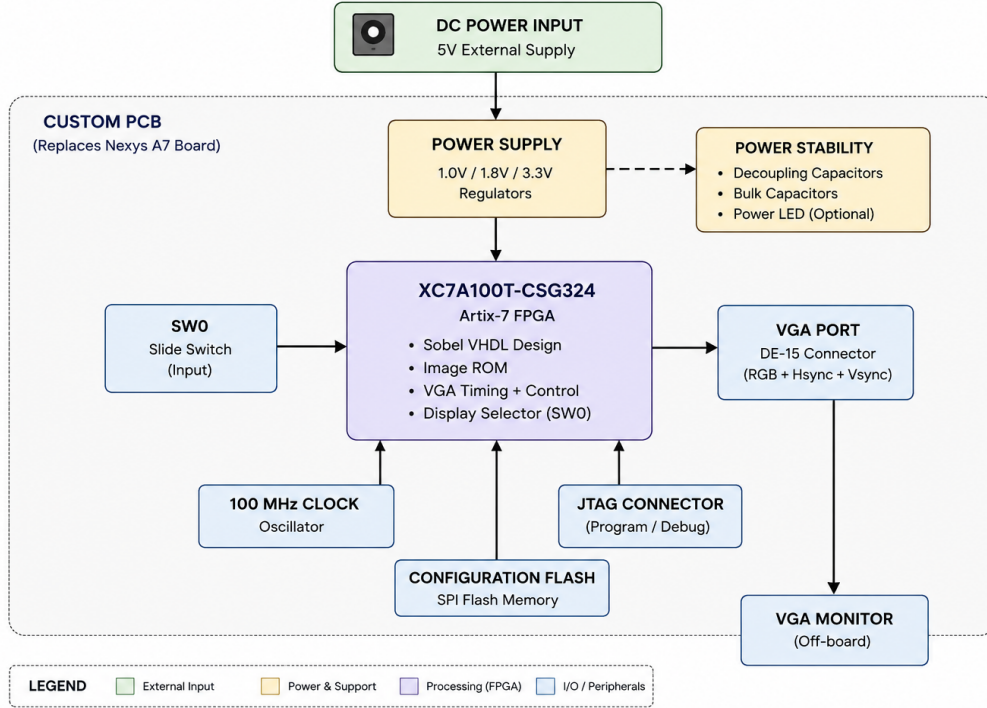


Fig. 3: Hardware Realization and Custom PCB block diagram.

logic. The custom board must recreate only the supporting hardware: power rails, clock, configuration, programming, switch input, and VGA output. This follows the laboratory method of analyzing project requirements, identifying peripherals, selecting components, preparing a modular schematic, completing floorplanning and routing, and finally generating manufacturing outputs.

The selected FPGA is the same XC7A100T-1CSG324C used on the development board. Reusing the device minimizes VHDL portability risk and preserves the known Artix-7 pin and resource environment. A 5-V input feeds three TPS62130-based buck stages for 1.0 V, 1.8 V, and 3.3 V. The 1.0-V rail supplies the core, 1.8 V supplies auxiliary/configuration

functions, and 3.3 V supplies I/O banks and external peripherals. The schematic also contains the DSC1001 100-MHz oscillator, an MT25QL128 configuration flash, a 10-pin JTAG connector, an SW0 slide switch, a DE-15 VGA connector, inductors, bulk capacitors, local capacitors, and resistor networks.

V. SCHEMATIC DESIGN

The schematic method is modular. The FPGA section groups the 324-ball symbol and bank power pins. The power section separates the three regulator channels. The clock/configuration section contains the oscillator, JTAG, and flash. The peripheral section contains SW0, VGA, and related passive components. Net labels are used to connect sheets without long crossing wires. Power pins, JTAG reference voltage, configuration

signals, and all VGA outputs must be checked against the XDC assignments and FPGA data sheet. The supplied `SchDoc` contains the design data, but no raster schematic export was included; therefore, Fig. 4 reserves the required paper location for an Altium screenshot.

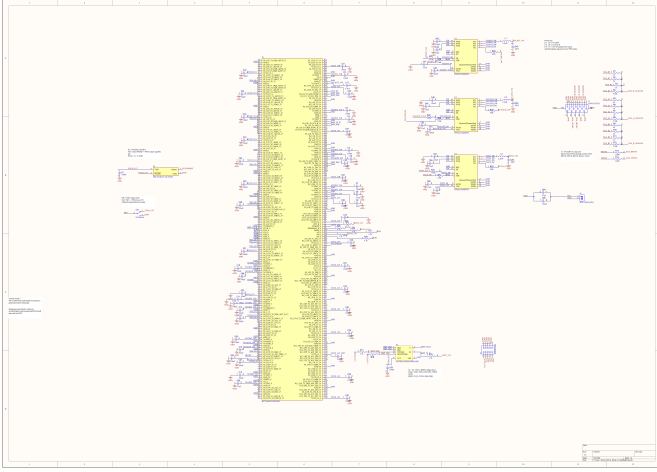


Fig. 4: Schematic Design for Custom PCB SchDoc.

VI. PCB LAYOUT AND OUTPUTS

Floorplanning places the FPGA centrally, the three regulator/inductor stages near the power entry, the oscillator close to a clock-capable FPGA ball, and the flash close to configuration pins. The VGA connector, JTAG header, power connector, and slide switch are positioned at board edges for access. Decoupling components are placed around the FPGA power pins, while power routes are kept wider than ordinary signals. The supplied manufacturing set contains top and bottom copper, four internal plane layers (GP1–GP4), solder masks, overlays, mechanical outline, and NC-drill data; this corresponds to a six-copper-layer output stack.

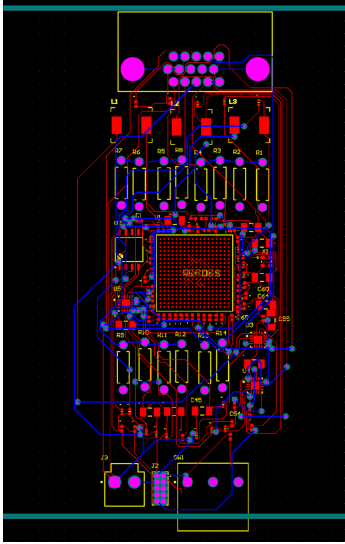


Fig. 5: 2-D layout Gerber files.

The exported BOM contains 13 line items and 109 component placements. Its main entries are summarized in Table III.

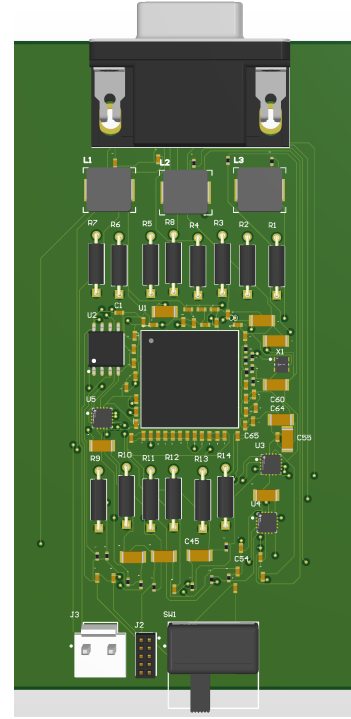


Fig. 6: Altium 3D PCB Design

The list includes 64 capacitors, 32 resistors, three inductors, three regulators, one FPGA, one 128-Mbit serial flash, one 100-MHz oscillator, three connectors, and one slide switch. The VGA connector is J1, JTAG is J2, power entry is J3, the FPGA is U1, flash is U2, regulators are U3–U5, and the oscillator is X1.

TABLE III: BOM Summary from the Supplied Export

Item	Designator	Qty.
6.8-nF 0402 capacitors	C1–C66 subset	53
10- μ F 1206 capacitors	C8–C64 subset	11
VGA DE-15 connector	J1	1
JTAG connector	J2	1
Power connector	J3	1
2.2- μ H inductors	L1–L3	3
475- Ω resistors	R1–R14	14
84.5-k Ω resistors	R16–R33	18
Slide switch	SW1	1
XC7A100T-CSG324 FPGA	U1	1
MT25QL128 flash	U2	1
TPS62130 regulators	U3–U5	3
100-MHz oscillator	X1	1
Total placements		109

Gerber and drill generation demonstrates the manufacturing-output workflow. The drill report lists 60 plated holes across six tool sizes. However, the supplied DRC report dated 27 July 2026 contains 0 warnings but 484 rule violations, including 124 unrouted-net, 90 clearance, 122 solder-mask-sliver, and 111 silk-to-solder-mask violations. Therefore, the board is an advanced educational layout and output package, but it is not yet ready for fabrication. The schematic should be exported, ERC reviewed, all nets routed, DRC rules matched to the selected manufacturer, polygons repoured, and Gerber/drill outputs regenerated before ordering.

VII. CONCLUSION

A ROM-based Sobel lane-edge detector was implemented on the Artix-7 XC7A100T and displayed through VGA. The four-file VHDL architecture separates timing, image storage, filtering, and top-level control. Upgrading the stored frame from 160×120 to 320×240 increases source pixels by four, improves visible edge detail, and reduces enlargement from $4 \times$ to $2 \times$. The cost is mainly on-chip memory: BRAM utilization rises from 5.93% to 17.78%, matching the uploaded Vivado graphs rounded from 6% to 18%. The upgraded version is the highest practical resolution completed in the project, while still retaining significant device headroom.

The FPGA-to-PCB phase converts the development-board prototype into a project-specific hardware architecture with the FPGA, three power rails, oscillator, flash, JTAG, SW0, VGA, and stabilization components. Actual Altium outputs provide a PCB document, 3-D view, BOM, Gerber layers, and drill data. The remaining work is to replace the schematic placeholder with an exported screenshot and close all DRC violations before fabrication. The combined VHDL and PCB workflow

demonstrates the full path from algorithm concept, simulation, synthesis, and hardware display to board-level implementation and manufacturing preparation.

VIII. TEAM CONTRIBUTIONS

Md Mostafizur Rahman completed the overall project concept, VHDL implementation, FPGA implementation, hardware configuration, documentation, and the main FPGA-to-PCB development. Deepak Kapil worked only on the testbench and verification. Turja Barua worked on PCB netlisting, PCB layout, Gerber and BOM. Md Mostafizur Rahman also assisted Turja Barua with the schematic, layout and routing.

REFERENCES

- [1] R. C. Gonzalez and R. E. Woods, *Digital Image Processing*, 4th ed. Pearson, 2018.
- [2] I. Sobel and G. Feldman, "A 3×3 isotropic gradient operator for image processing," Stanford Artificial Intelligence Project, 1968/1973.
- [3] AMD, "Artix-7 FPGAs Data Sheet: DC and AC Switching Characteristics," DS181.
- [4] Digilent, "Nexys A7 FPGA Board Reference Manual."
- [5] AMD, "7 Series FPGAs PCB Design Guide," UG483.